



以RAFT模型代码为例的光流相关问题总结



人工菜鸡
学生狗

41 人赞同了该文章

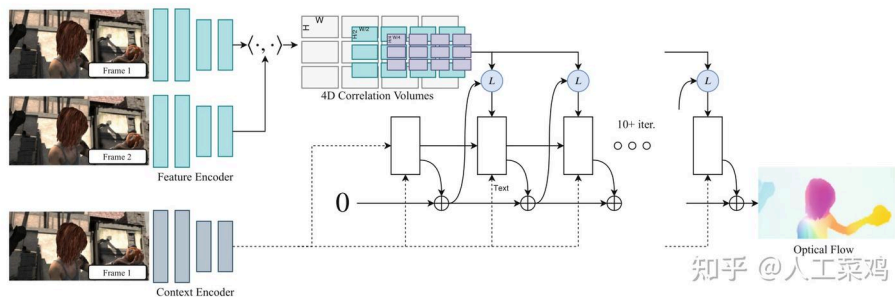
以raft为例求光流的完整过程和代码解释（backward warping的具体操作与光流的方向问题，cost volume的构建与搜索，双线性插值原理，前后向warping的区别问题）

RAFT: Recurrent All-Pairs Field Transforms for Optical Flow

论文：[arxiv.org/pdf/2003.12033...](https://arxiv.org/pdf/2003.12033)

源码：[github.com/princeton-vl...](https://github.com/princeton-vl/)

inference：用训练好的模型对连续图像帧求光流。



把模型放到GPU上：

```
# model = RAFT.cuda() 单卡
model = torch.nn.DataParallel(RAFT(args)) # 多卡
```

注意图像帧的命名前面要用zfill函数补0，不然读取顺序是乱的；对图像帧两两读取的方法：用zip函数把图片第0个到倒数第一个，第1个到最后一个打包，然后迭代，每次就可以取出相邻的两帧。

```
images = sorted(images)
for imfile1, imfile2 in zip(images[:-1], images[1:]):
    image1 = load_image(imfile1)
    image2 = load_image(imfile2)
```

知乎 @人工菜鸡

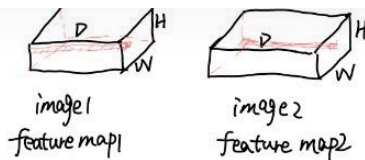
本例image的shape是：Tensor[1,3,1280,720]

进入模型后流程：

1. 两个image归一化到[-1, 1]：

```
image2 = 2 * (image2 / 255.0) - 1.0
```

- 两帧进入feature network，把两幅在第0维度上concat(Tensor[2,3,1280,720])，进入网络，相当于同时求两帧的特征图。得到[1,256,160,90]的两个特征图，channel数为256，height为160，width为90。
- 用得到的特征图构建correlation volume：
correlation volume的原理介绍：



对于RGB图像，每个像素位置的维数由3维变为了D维，该位置上这个D维的向量就是这个像素位置的特征向量，即每个feature map

有 $H \times W$ 个特征向量，每个特征向量都是D维的。

归一化后的两个向量点积， $\rightarrow \cdot$ ，结果越接近1，则两个向量越相似。

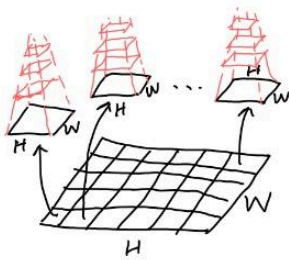
由这个原理，用各位置特征向量的点积值表示像素的匹配程度

如：shape (2, 2, 3) 的 a, b，把第一维看作 channel D

a: $\begin{bmatrix} \text{channel 1} & \text{channel 2} \\ \begin{bmatrix} 0.1 & 0.6 & 0.3 \\ 0.2 & 0.9 & 0.4 \end{bmatrix} & \begin{bmatrix} 0.1 & 0.6 & 0.3 \\ 0.2 & 0.9 & 0.4 \end{bmatrix} \end{bmatrix}$ 则 a 的 (0,1) 位置像素的特征向量为 $\begin{pmatrix} 0.6 \\ 0.6 \end{pmatrix}$
 b: $\begin{bmatrix} \begin{bmatrix} 0.5 & 0.8 & 0.3 \\ 0.4 & 0.8 & 0.6 \end{bmatrix} & \begin{bmatrix} 0.8 & 0.6 & 0.8 \\ 0.5 & 0.7 & 0.3 \end{bmatrix} \end{bmatrix}$ b 的 (0,1) 位置像素的特征向量为 $\begin{pmatrix} 0.8 \\ 0.6 \end{pmatrix}$
 两个像素的相似程度为 $\begin{pmatrix} 0.6 \\ 0.6 \end{pmatrix} \cdot \begin{pmatrix} 0.8 \\ 0.6 \end{pmatrix} = 0.8$
 (0.6) 归一化；即除以模长变为： $\begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix}$
 (0.8) 归一化变为： $\begin{pmatrix} \frac{4}{5} \\ \frac{3}{5} \end{pmatrix}$

相似度则为 $\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \cdot \begin{pmatrix} \frac{4}{5} \\ \frac{3}{5} \end{pmatrix} = \frac{7}{5\sqrt{2}}$ ，很接近1，表示两者很相似。

如果两个相同的特征向量点积，如 $\begin{pmatrix} \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \end{pmatrix} \cdot \begin{pmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{pmatrix} = 1$ ，则值为1，表示完全相同
 feature map 1 的每个位置的特征向量都会和 feature map 2 的每个位置的特征向量计算一次相似度，所以共得到 $H \times W \times H \times W$ 个相似度。



论文还对 volume 的后两维做了三次 pooling 构成了多层金字塔结构使搜索范围有大有小

$$\begin{aligned}
 \text{fmap1} &: \left[\begin{bmatrix} 0.1 & 0.6 & 0.3 \\ 0.2 & 0.9 & 0.4 \end{bmatrix} \begin{bmatrix} 0.1 & 0.8 & 0.7 \\ 0.2 & 0.9 & 0.4 \end{bmatrix} \right] \text{fmap2} : \left[\begin{bmatrix} 0.5 & 0.8 & 0.3 \\ 0.4 & 0.6 & 0.6 \end{bmatrix} \begin{bmatrix} 0.5 & 0.7 & 0.3 \end{bmatrix} \right] \\
 \text{fmap1.view}(1, 2, 6) &\rightarrow \begin{bmatrix} 0.1 & 0.6 & 0.3 & 0.2 & 0.9 & 0.4 \\ 0.1 & 0.8 & 0.7 & 0.2 & 0.9 & 0.4 \end{bmatrix} f_1 (1, 2, 6) \\
 \text{同理 fmap2.view}(1, 2, 6) &\rightarrow \begin{bmatrix} 0.5 & 0.8 & 0.3 & 0.4 & 0.8 & 0.6 \\ 0.8 & 0.6 & 0.8 & 0.5 & 0.7 & 0.3 \end{bmatrix} f_2 \\
 f_1.\text{transpose}(1, 2) &\rightarrow \begin{bmatrix} 0.1 & 0.1 \\ 0.6 & 0.8 \\ 0.3 & 0.7 \\ 0.2 & 0.2 \\ 0.9 & 0.9 \\ 0.4 & 0.4 \end{bmatrix} f_1 (1, 6, 2) \\
 \text{corr} = \text{matmul}(f_1, f_2) &= \begin{pmatrix} 0.1 & 0.1 \\ 0.6 & 0.8 \\ 0.3 & 0.7 \\ 0.2 & 0.2 \\ 0.9 & 0.9 \\ 0.4 & 0.4 \end{pmatrix} \begin{pmatrix} 0.5 & 0.8 & 0.3 & 0.4 & 0.8 & 0.6 \\ 0.8 & 0.6 & 0.8 & 0.5 & 0.7 & 0.3 \end{pmatrix} \\
 &= \begin{pmatrix} 0.13 & 0.14 & 0.11 & 0.09 & 0.15 & 0.09 \\ 0.94 & 0.96 & 0.82 & 0.64 & 1.04 & 0.6 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \end{pmatrix} \quad \begin{matrix} \text{每一列都是特征向量} \\ \text{没归一化} \end{matrix} \\
 \text{corr} = \text{corr.view}(1, 2, 3, 1, 2, 3) & \quad \begin{matrix} \text{每一行都是特征向量} \\ \text{B H W 1 H W} \end{matrix} \\
 \text{corr} = \text{corr.mul}(\frac{1}{\text{channel}}) & \quad \text{类似于归一化, 没有超过1的值了, 原理存疑} \\
 \text{corr.reshape}(b \times h \times w, 1, h, w) & : (6, 1, 2, 3)
 \end{aligned}$$

知乎 @人工菜鸡

代码:

```

def corr(fmap1, fmap2):
    batch, dim, ht, wd = fmap1.shape
    fmap1 = fmap1.view(batch, dim, ht*wd)
    fmap2 = fmap2.view(batch, dim, ht*wd)

    corr = torch.matmul(fmap1.transpose(1, 2), fmap2)
    corr = corr.view(batch, ht, wd, 1, ht, wd)
    # return corr / torch.sqrt(torch.tensor(dim).float())
    return corr.mul_(1.0/torch.sqrt(torch.tensor(dim).float()))

```

然后把corr reshape成[batch*h1*w1,dim,h2,w2], 即[14400, 1, 160, 90], 再下采样3次, 得到4层的correlation金字塔. ([14400, 1, 160, 90], [14400, 1, 80, 45] [14400, 1, 40, 22] [14400, 1, 20, 11])

```

corr = corr.reshape(batch*h1*w1, dim, h2, w2)

self.corr_pyramid.append(corr)

for i in range(self.num_levels-1):
    corr = F.avg_pool2d(corr, 2, stride=2)
    self.corr_pyramid.append(corr)

```

知乎 @人工菜鸡

```
cnet = self.cnet(image1)  cnet: tensor([[[[ 2.351
net, inp = torch.split(cnet, [hdim, cdim], dim=1)
net = torch.tanh(net)  # 1,128,160,90
inp = torch.relu(inp)  # 1,128,160,90
```

知乎 @人工菜鸡

- 然后是光流的具体表示，用torch.meshgrid函数画网格，光流的计算为： $\text{flow} = \text{coords1} - \text{coords0}$ ，注意，是后面帧减前面帧的。现在相当于有了初始的光流了，接下来就要做warping了。coords第一维的2表示光流的x和y(上下)两个方向。

```
N, C, H, W = img.shape
coords0 = coords_grid(N, H//8, W//8, device=img.device)
coords1 = coords_grid(N, H//8, W//8, device=img.device)
```

```
def coords_grid(batch, ht, wd, device):  device: cuda:0  batch: 1  ht: 160  wd: 90
    coords = torch.meshgrid(torch.arange(ht, device=device), torch.arange(wd, device=device))
    # coords是tuple, 由两个(160,90)的tensor组成, 初始值无所谓
    coords = torch.stack(coords[::-1], dim=0).float()  # 2,160,90
    return coords[None].repeat(batch, 1, 1, 1)  # 复制batch个网格
```

- 由于在Corrblock*中定义了__call__魔法方法，则Corrblock的类对象都是callable的，即可以通过实例对象后面加括号的方式调用__call__函数：即corr_fn(coords1)调用了__call__函数。主要需要看懂的就是__call__函数和里面bilinear_sampler*函数的作用。

```
flow_predictions = []  flow_predictions: [tensor([[[[-4.6647e-03, -2.8306e-
for itr in range(iters):  itr: 1
    coords1 = coords1.detach()
    corr = corr_fn(coords1)  # index correlation volume (1,324,160,90)  cor
    flow = coords1 - coords0  flow: tensor([[[[0., 0., 0., ..., 0., 0., 0.
    with autocast(enabled=self.args.mixed_precision):
        net, up_mask, delta_flow = self.update_block(net, inp, corr, flow)
    #  $F(t+1) = F(t) + \Delta(t)$ 
    coords1 = coords1 + delta_flow
    # upsample predictions
    if up_mask is None:
        flow_up = upflow8(coords1 - coords0)  flow_up: tensor([[[[-4.6647e-
    else:
        flow_up = self.upsample_flow(coords1 - coords0, up_mask)
    flow_predictions.append(flow_up)

if test_mode:
    return coords1 - coords0, flow_up

return flow_predictions
```

知乎 @人工菜鸡

```

coords = coords.permute(0, 2, 3, 1) # 变为[1, 100, 70, 2]
batch, h1, w1, _ = coords.shape
out_pyramid = []
for i in range(self.num_levels):
    corr = self.corr_pyramid[i]
    dx = torch.linspace(-r, r, 2*r+1, device=coords.device)#[ -4,-3,-2,-1,0,1,2,3,4]
    dy = torch.linspace(-r, r, 2*r+1, device=coords.device)#[ -4,-3,-2,-1,0,1,2,3,4]
    delta = torch.stack(torch.meshgrid(dy, dx), axis=-1)#[0,9,2]
    centroid_lvl = coords.reshape(batch*h1*w1, 1, 1, 2) / 2**i#[14400,1,1,2]
    delta_lvl = delta.view(1, 2*r+1, 2*r+1, 2)#[1,9,9,2]
    coords_lvl = centroid_lvl + delta_lvl #[14400,9,9,2]
    corr = bilinear_sampler(corr, coords_lvl)
    corr = corr.view(batch, h1, w1, -1)
    out_pyramid.append(corr)
out = torch.cat(out_pyramid, dim=-1)
return out.permute(0, 3, 1, 2).contiguous().float()

```

知乎 @人工菜鸡

```

def bilinear_sampler(img, coords, mode='bilinear', mask=False):
    """ Wrapper for grid_sample, uses pixel coordinates """
    H, W = img.shape[-2:]
    xgrid, ygrid = coords.split([1,1], dim=-1)
    xgrid = 2*xgrid/(W-1) - 1
    ygrid = 2*ygrid/(H-1) - 1

    grid = torch.cat([xgrid, ygrid], dim=-1)
    img = F.grid_sample(img, grid, align_corners=True)

    if mask:
        mask = (xgrid > -1) & (ygrid > -1) & (xgrid < 1) & (ygrid < 1)
        return img, mask.float()

    return img

```

知乎 @人工菜鸡

代码解释：

则相应的 corr 是 $(48, 1, 8, 6)$ 的

$$\left(\begin{pmatrix} 0.1 & 0.2 & 1 & 1 \\ 0.4 & 0.6 & 1 & 1 \\ \vdots & \vdots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots \end{pmatrix} \begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \right) \dots \dots \dots \left(\begin{pmatrix} 1 \\ 1 \\ 1 \\ 1 \end{pmatrix} \right)$$

1帧第1个像素和2帧所有像素的匹配度

$\text{centroid_lvl} : ((5,1)(4,2) \dots (6,1))$

质心 48个 $1 \times 1 \times 2$, 即表示在每个像素处开始搜索

delta_lvl 变成 $(1, 5, 5, 2)$, 看上去即 delta , $\text{coords_lvl} = \text{centroid_lvl} + \text{delta_lvl}$

即以每个像素为起始点向上向下向左向右分别走2步得到的范围

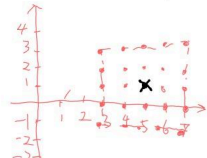
不同维的加法会触发 tensor 的广播机制 $(48, 1, 1, 2) + (1, 5, 5, 2)$ 可广播

可广播原则: 两 tensor 从尾部维度开始: ①或者相等 ②或者其中一个 tensor 维度为1 ③或者其中一个 tensor 不存在这个维度 广播后

$$\left(\begin{pmatrix} 5 & 5 & 5 & 5 \\ 5 & 5 & 5 & 5 \\ 5 & 5 & 5 & 5 \\ 5 & 5 & 5 & 5 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \end{pmatrix} \right) + \left(\begin{pmatrix} 4 & 4 & 4 & 4 \\ 4 & - & - & - \\ 4 & - & - & - \\ 4 & - & - & - \end{pmatrix} \begin{pmatrix} 2 & 2 & 2 & 2 \\ 2 & - & - & - \\ 2 & - & - & - \\ 2 & - & - & - \end{pmatrix} \right) \dots \dots \dots + \left(\begin{pmatrix} -2 & -2 & -2 & -2 \\ -2 & -1 & 0 & 1 & 2 \\ -2 & -1 & 0 & 1 & 2 \\ -2 & -1 & 0 & 1 & 2 \\ -2 & -1 & 0 & 1 & 2 \end{pmatrix} \right)$$

$$= \left(\begin{pmatrix} 3 & 3 & 3 & 3 \\ 4 & 4 & 4 & 4 \\ 5 & 5 & 5 & 5 \\ 6 & 6 & 6 & 6 \\ 7 & 7 & 7 & 7 \end{pmatrix} \begin{pmatrix} -1 & 0 & 1 & 2 & 3 \\ -1 & 0 & 1 & 2 & 3 \\ -1 & 0 & 1 & 2 & 3 \\ -1 & 0 & 1 & 2 & 3 \\ -1 & 0 & 1 & 2 & 3 \end{pmatrix} \right) \dots \dots \dots$$

x坐标 y坐标 (x,y写反了, 意思一样)



所以 coords_lvl 得到的是 第2帧每像素移动后的坐标周围以 r 为半径的所有坐标, 这些坐标为匹配的潜在位置。

$H=8$ $W=6$ $\rightarrow$ grid 正坐标 $(48, 5, 5, 1)$ $\rightarrow$ grid 归一化

$xgrid = 2 \times xgrid / (W-1) - 1$ x 用 W 归一化, 因为 x 取值按理都小于 W

$ygrid = 2 \times ygrid / (H-1) - 1$ y 归一化同理 归一化完再合起来为 $grid(48, 5, 5, 2)$

最重要的一步: $F.grid_sample(corr, grid)$

torch 源码里对 $grid_sample$ 函数有非常详尽的注释, 概括如下:

4维情况下, 输入 (N, C, H, W) , $grid(N, H, W, 2)$, 输出 (N, C, H, W) ,

对于输出的每个位置 $[n, :, h, w]$, 是由 $grid[n, h, w]$ 指定的 x, y 坐标去输入的对应该位置插值得到的。 $grid$ 的值要提前用输入的 $size$ 作 *normalization*, 所以 $grid$ 的大多数值在 $[-1, 1]$ 之间, 如 $x=-1, y=-1$ 是输入的左上角; $x=1, y=1$ 是输入的右下角。

超出范围的值由 $padding_mode$ 参数决定

是补零还是用其他方法。插值方法有双线性插值、最近邻插值。

输出: $(48, 1, 5, 5)$

$$\begin{pmatrix} 0 & 0 & 0 & 0 & 0 \\ 4.1e^{-8} & 5.9e^{-8} & 4.1e^{-8} & 0 & 0 \\ 0.69 & 1 & 0.69 & 0 & 0 \\ 0.31 & 0.89 & 0.1 & 0 & 0 \\ 0.8 & 0.5 & 0.6 & 0 & 0 \end{pmatrix} \dots$$

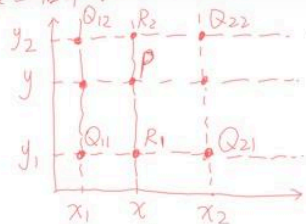
48个每个像素位置与其周围潜在位置的匹配度

为什么要插值? 因为 $coords$ 里的值可能是小数, 使 $coords_lut$ 里也是小数, 去 $corr$ 相关性表里找该像素与其周围位置的匹配度时, 小数位置的匹配度就由其周围的4个整数位置插值得到。

知乎 @人工菜鸡

双线性插值推导:

点的灰度值估计该点的灰度值



已知 $Q_{11}, Q_{12}, Q_{21}, Q_{22}$ 坐标，求 P 坐标处函数值

$$\begin{aligned} (x_2 - x_1)(y_2 - y_1) &= (x_2 - x_1)(y_2 - y_1) \\ x_2 y - x_2 y_1 - x_1 y + x_1 y_1 &= \\ x_2 y - x_2 y_1 - x_1 y + x_1 y_1 &= \\ (x_2 - x_1)y &= x_2 y_1 + x_1 y_2 - x_1 y_1 - x_1 y_2 \end{aligned}$$

①在 x 方向线性插值， y_1 固定， y_2 固定， y 固定

$$\frac{x_2 - x_1}{y_2 - y_1} = \frac{x - x_1}{y - y_1} \Rightarrow y = \frac{x_2 - x_1}{x_2 - x_1} y_1 + \frac{x - x_1}{x_2 - x_1} y_2$$

$$\therefore f(R_1) = \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21}) \quad f(R_2) = \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22})$$

同理在 y 方向插值：

$$f(P) = \frac{y_2 - y}{y_2 - y_1} f(R_1) + \frac{y - y_1}{y_2 - y_1} f(R_2) \quad \because x_2 - x_1, y_2 - y_1 \text{ 都为 } 1$$

$$\text{整理: } f(P) = (y_2 - y)(x_2 - x)f(Q_{11}) + (y_2 - y)(x - x_1)f(Q_{21}) + (y - y_1)(x_2 - x)f(Q_{12}) + (y - y_1)(x - x_1)f(Q_{22})$$

把每个 $f(\cdot)$ 前的系数看作权重，则每个整数点的权重和这个整数点的对角点与待求点的坐标差的乘积有关

如 $f(Q_{11})$ 的权重与 $f(Q_{22})$ 的坐标有关，也可写成

$$f(P) = \sum_q \max(0, 1 - |q_x - p_x|) \max(0, 1 - |q_y - p_y|) f(q)$$

要用 1 减因为是对角，而 $y_2 - y_1 = 1, x_2 - x_1 = 1$ 知乎 @人工菜鸡

经过一次 `__call__` 函数后得到的 `out_pyramid` 是在多尺度下寻找匹配度的金字塔，最终 `out` 的维数为 (1,160,90,324)

```
> out = {Tensor: (1, 160, 90, 324)} t
> out_pyramid = {list: 4} [tensor([[[[
> 0 = {Tensor: (1, 160, 90, 81)} t
> 1 = {Tensor: (1, 160, 90, 81)} t
> 2 = {Tensor: (1, 160, 90, 81)} t
> 3 = {Tensor: (1, 160, 90, 81)} t
```

• 然后GRU迭代：

```
class BasicUpdateBlock(nn.Module):
    def __init__(self, args, hidden_dim=128, input_dim=128):
        super(BasicUpdateBlock, self).__init__()
        self.args = args
        self.encoder = BasicMotionEncoder(args)
        self.gru = SepConvGRU(hidden_dim=hidden_dim, input_dim=128+hidden_dim)
        self.flow_head = FlowHead(hidden_dim, hidden_dim=256)

        self.mask = nn.Sequential(
            nn.Conv2d(128, 256, 3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(256, 64*9, 1, padding=0))

    def forward(self, net, inp, corr, flow, upsample=True):
        motion_features = self.encoder(flow, corr) # 结合光流和相关性图提取特征
        inp = torch.cat([inp, motion_features], dim=1) # 连接 Context Encoder 提取的特征和上面提取的特征

        net = self.gru(net, inp) # GRU 迭代，更新隐状态 net
        delta_flow = self.flow_head(net) # 由隐状态得到光流残差

        # scale mask to balance gradients
        mask = .25 * self.mask(net) # 由隐状态得到上采样 mask
        return net, mask, delta_flow
```

知乎 @人工菜鸡


```

super(SepConvGRU, self).__init__()
self.convz1 = nn.Conv2d(hidden_dim+input_dim, hidden_dim, (1,5), padding=(0,2))
self.convr1 = nn.Conv2d(hidden_dim+input_dim, hidden_dim, (1,5), padding=(0,2))
self.convq1 = nn.Conv2d(hidden_dim+input_dim, hidden_dim, (1,5), padding=(0,2))

self.convz2 = nn.Conv2d(hidden_dim+input_dim, hidden_dim, (5,1), padding=(2,0))
self.convr2 = nn.Conv2d(hidden_dim+input_dim, hidden_dim, (5,1), padding=(2,0))
self.convq2 = nn.Conv2d(hidden_dim+input_dim, hidden_dim, (5,1), padding=(2,0))

# 将 3x3 卷积替换成 1x5 和 5x1 的两次卷积，在不提高参数量的情况下增大感受野
def forward(self, h, x):
    # horizontal
    hx = torch.cat([h, x], dim=1)
    z = torch.sigmoid(self.convz1(hx))
    r = torch.sigmoid(self.convr1(hx))
    q = torch.tanh(self.convq1(torch.cat([r*h, x], dim=1)))
    h = (1-z) * h + z * q

    # vertical
    hx = torch.cat([h, x], dim=1)
    z = torch.sigmoid(self.convz2(hx))
    r = torch.sigmoid(self.convr2(hx))
    q = torch.tanh(self.convq2(torch.cat([r*h, x], dim=1)))
    h = (1-z) * h + z * q

    return h

```

知乎 @人工菜鸡

FlowHead是为了把最后输出变为2维光流的简单网络。

GRU公式

$$\hat{c}^{<t>} = g(W_c[\Gamma_r * c^{<t-1>}, x^{<t>}] + b_c)$$

$$\Gamma_u = \sigma(W_u[c^{<t-1>}, x^{<t>}] + b_u)$$

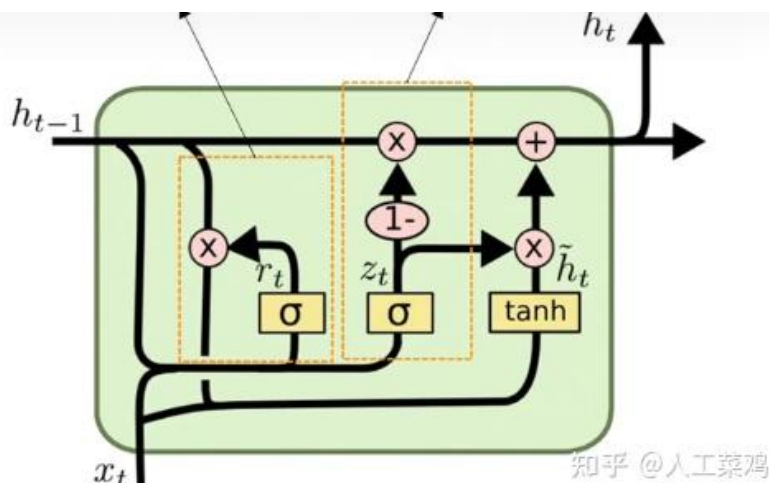
$$\Gamma_r = \sigma(W_r[c^{<t-1>}, x^{<t>}] + b_r)$$

$$c^{<t>} = \Gamma_u * \hat{c}^{<t>} + (1 - \Gamma_u) * c^{<t-1>}$$

$$a^{<t>} = c^{<t>}$$

$$\hat{y}^{<t>} = g(W_y a^{<t>} + b_y)$$

知乎 @人工菜鸡



$c^{<t>} = c^{<t>}$ 表示状态向量也是记忆单元, 保存之前的状态
 更新门 r_t 和重置门 z_t 都是经过 $\sigma(\cdot)$ sigmoid 函数后 0 到 1 之间的权重向量
 r_t 决定利用上一个时刻的记忆 $c^{<t-1>}$ 的程度, 得到这一时刻的暂时记忆 $\hat{c}^{<t-1>}$
 r_t 控制这一时刻受前-时刻的影响程度, r_t 为 0 时, $c^{<t>} = c^{<t-1>}$
 r_t 为 1 时, $c^{<t>} = \hat{c}^{<t>}$

知乎 @人工菜鸡

$$z_t = \sigma(\text{Conv}_{3 \times 3}([h_{t-1}, x_t], W_z))$$

$$r_t = \sigma(\text{Conv}_{3 \times 3}([h_{t-1}, x_t], W_r))$$

$$\tilde{h}_t = \tanh(\text{Conv}_{3 \times 3}([r_t \odot h_{t-1}, x_t], W_h))$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

知乎 @人工菜鸡

for 循环迭代 10 次得到 refine 后的最终光流。

上采样:

2 Upsampling Module

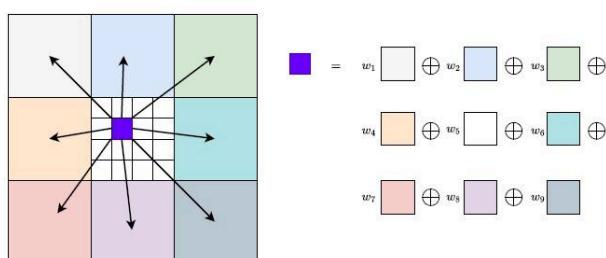
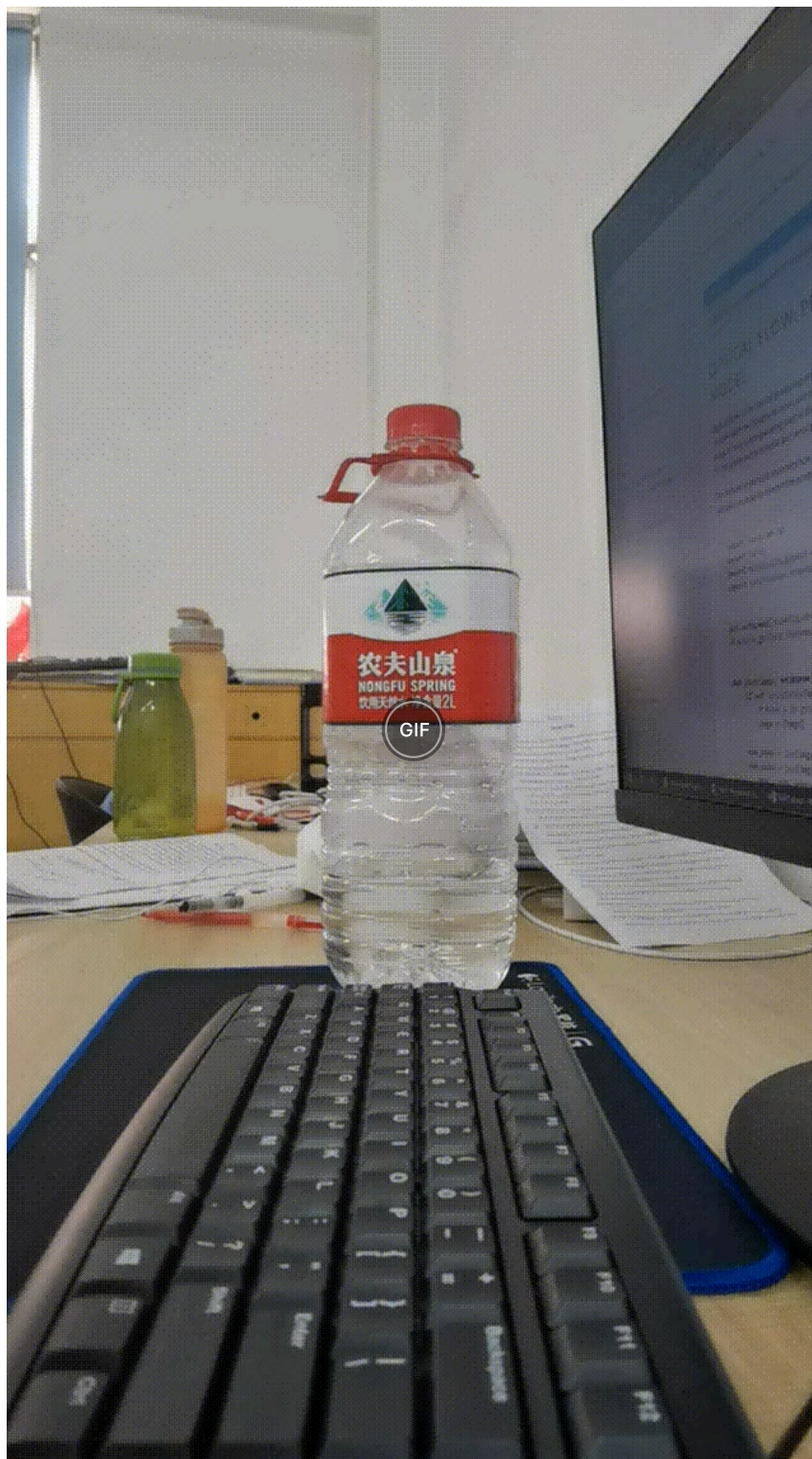
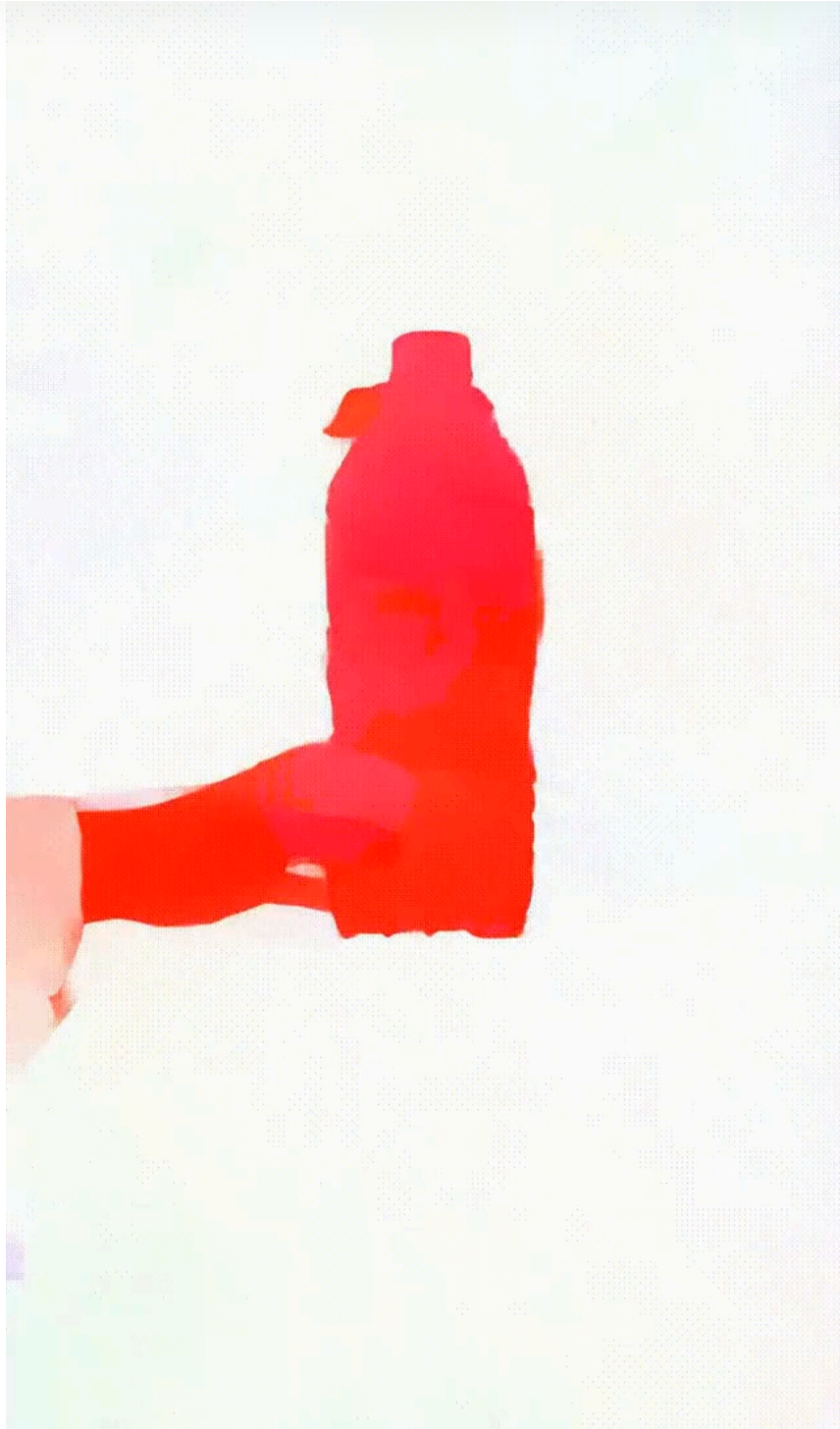


Fig. 2: Illustration of the upsampling module. Each pixel of the high resolution flow field (small boxes) is taken to be the convex combination of its 9 coarse resolution neighbors using weights predicted by the network. 知乎 @人工菜鸡

1. 总结: coords1 相当于经过了光流 warp 之后的第二帧, 接受了 coords 和 grid 的 bilinear_sampler 函数相当于隐含的 backward warping。只不过插的值不是像素值而是匹配度(?)。前向 warping 和

3. inference效果:





参考: [图像处理基础（十八）光流中的 warp 与遮挡检测 - 知乎 \(zhihu.com\)](#)

[光流warp原理 TensorFlow dense_image_warp源码解析 - 简书 \(jianshu.com\)](#)

[Pytorch光流warp函数解读（grid_sample函数的使用） - 知乎 \(zhihu.com\)](#)

[\(1条消息\) 图像变换——向前映射和向后映射_前向映射和后向映射_薇洛的打火机的博客-CSDN博客](#)

[\(1条消息\) 图像的前向和后向映射_前向映射_隔壁的二大爷的博客-CSDN博客](#)

[\(1条消息\) ECCV 2020 Best Paper: RAFT 光流检测代码详解_raft光流代码__Suraimu_的博客-CSDN博客](#)

[RNN、GRU、LSTM公式推导和矩阵变化_哔哩哔哩_bilibili](#)

光流 深度学习 (Deep Learning) 图像处理

▲ 赞同 41 ▼ ● 9 条评论 ↗ 分享 ❤ 喜欢 ★ 收藏 📄 申请转载 ...

写下你的评论...

9 条评论

默认 最新



xixishui

写的很详细，感谢

2023-06-30 · 安徽

● 回复 ❤ 1



逆天用户

超级感谢!



2024-09-04 · 英国

● 回复 ❤ 喜欢



Su66688

有做立体匹配的吗🙄

2024-08-13 · 辽宁

● 回复 ❤ 喜欢



secondchance 🍷

Inference 比较大的就是变化位移比较大的对吗

2024-07-03 · 北京

● 回复 ❤ 喜欢



fly

好tm有实力啊

2023-11-27 · 辽宁

● 回复 ❤ 喜欢



人类装逼精华

请问，这个查表是对于每个像素的特征在一定范围内，找下一帧对应的像素特征吗？就是比如说下一帧可能范围，我把全部他全部拉出来，然后用一个max返回出最大值所在的坐标，然后这个坐标减去这一帧的坐标就可以得到光流。是这个意思吗？那么这些比较大小的操作是都可微的吗？

2023-11-20 · 沙特阿拉伯

● 回复 ❤ 喜欢



人工菜鸡 作者

好像不是直接取最大吧，应该是在网络里传播了？我也搞不懂了

2023-12-14 · 日本

● 回复 ❤ 喜欢



人类装逼精华 · 人工菜鸡

好的，还是谢谢

2023-12-14 · 沙特阿拉伯

● 回复 ❤ 喜欢

展开其他 1 条回复 >

写下你的评论...

推荐阅读

光场传输矩阵成像仿真模型构建

摘抄自博士论文第3.1节 (i) 构建光场传输矩阵仿真模型 假设物平面上的点光源阵列为 80×80 pixels，若能分别获取这 80×80 个点光源透过散射介质后在成像面处的点扩散函数 (PSF)，就能获取成...

庄斌

发表于散射成像

一篇文章带你搞懂全局光照算法 RSM, SSDO, 从理论到实现

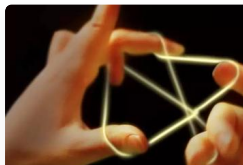
上篇文章我们实现了AO环境光遮蔽，解决了远处的间接光以及其遮蔽问题。但AO不能解决近处物体反射的间接光，所以今天实现的算法RSM (reflective shadow map) 解决了这个问题。但却不能解决AO...

Mr.Meeseeks



光流法研究笔记1(附送测试matlab源码)

奔跑的小伟



其实，光学优化中的光场以很简单

lilac的老父亲

